INTEGRATION TESTING – StreetRace Manager

2.1 System Overview

The StreetRace Manager system is a command-line application designed to manage underground street racing operations. The system was developed using a modular approach, where each component was implemented independently and later integrated.

The goal of integration testing is to verify that these modules interact correctly and follow the defined business rules.

Modules Implemented

Required Modules

1. **Registration Module**
   Responsible for registering new crew members by storing their name and role.
2. **Crew Management Module**
   Manages crew member roles (driver, mechanic, strategist) and assigns skill levels.
3. **Inventory Module**
   Maintains records of cars and cash balance. Also tracks whether vehicles are damaged.

4. **Race Management Module**
   Handles race creation and ensures only valid drivers and functional cars are used.
5. **Results Module**
   Records race outcomes and updates the inventory's cash balance.
6. **Mission Planning Module**
   Assigns missions and verifies that required crew roles are available before execution.

Additional Modules

7. **Vehicle Maintenance Module**
   Handles repair of damaged vehicles to ensure they can be reused in races.
8. **Leaderboard Module**
   Ranks crew members based on skill levels, enabling performance comparison.

Module Interactions

The system enforces several business rules through interactions between modules:

- A crew member must be registered before role assignment.

- Only crew members with the role "driver" can participate in races.
- Cars must not be damaged before being used in races.
- Race results update the inventory’s cash balance.
- Missions require specific roles, and execution is blocked if roles are missing.
- Damaged vehicles require repair before reuse.

These interactions ensure consistency and correctness across the system.

2.2 Call Graph

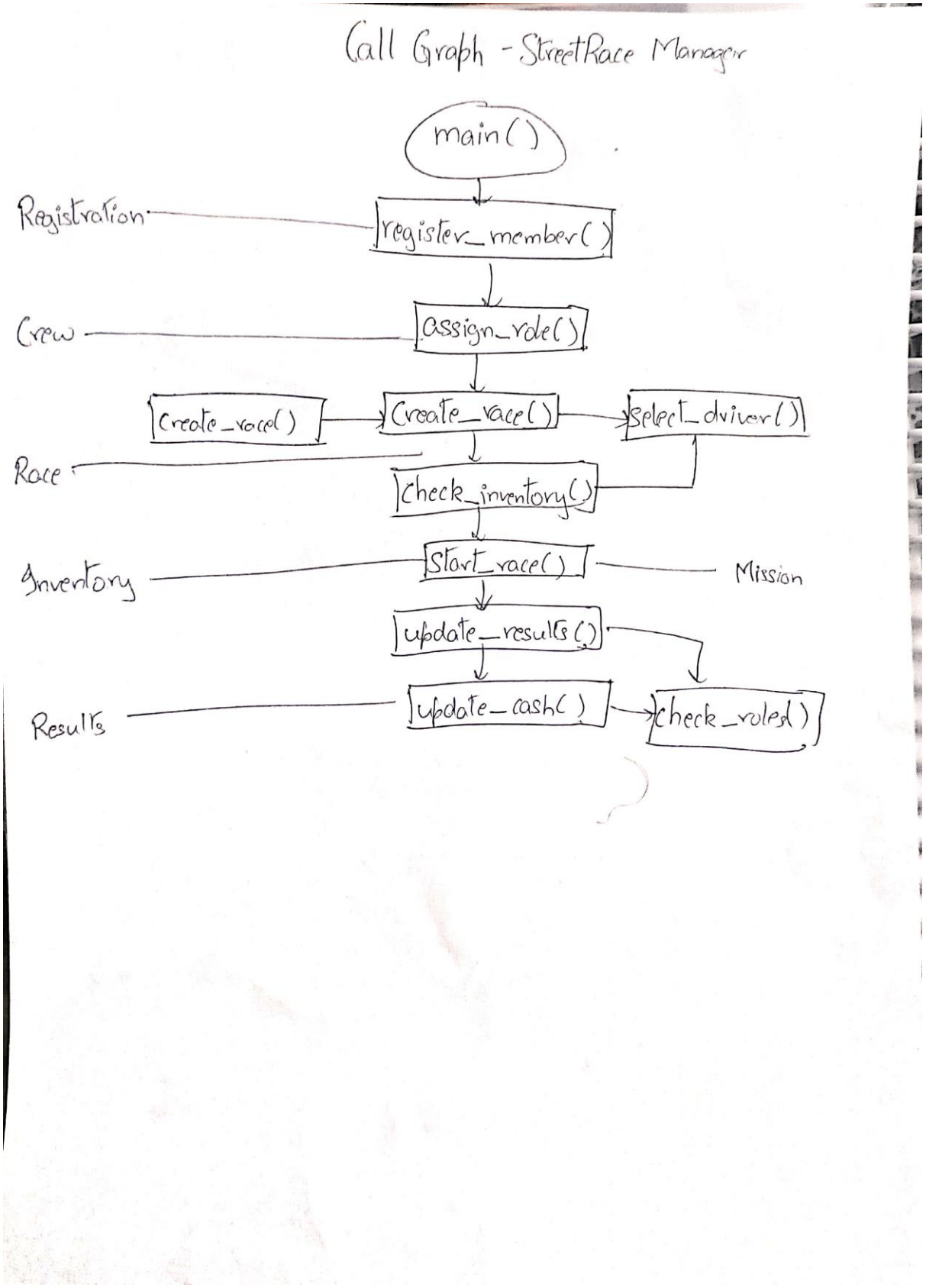
The Call Graph illustrates how functions across modules interact during execution.

Description

Execution begins in the main function. A crew member is registered, followed by role assignment and skill updates. The race module then selects eligible drivers and cars, verifying inventory conditions before starting a race.

After the race, results are recorded, and inventory cash is updated. Missions are then assigned, ensuring required roles are present. Additional modules such as

maintenance and leaderboard support extended functionality.



2.3 Integration Test Design

Integration test cases were designed to validate interactions between modules rather than individual functionality.

Test Case 1: Registering a Driver and Entering Race

- **Modules Involved:** Registration, Race
- **Scenario:** A crew member is registered as a driver and entered into a race
- **Expected Result:** Race is successfully created
- **Actual Result:** Race created successfully
- **Explanation:** Ensures proper data flow between registration and race modules

---

Test Case 2: Invalid Driver Entry

- **Modules Involved:** Registration, Race
- **Scenario:** A crew member with a non-driver role attempts to enter a race
- **Expected Result:** Error raised
- **Actual Result:** ValueError raised
- **Explanation:** Ensures role-based restrictions are enforced

---

Test Case 3: Race Results Update Inventory

- **Modules Involved:** Race, Results, Inventory
- **Scenario:** A race is completed and prize money is awarded
- **Expected Result:** Inventory cash increases
- **Actual Result:** Cash updated correctly
- **Explanation:** Validates correct interaction between results and inventory

---

Test Case 4: Mission Role Validation Failure

- **Modules Involved:** Mission, Crew
- **Scenario:** A mission is assigned without required roles
- **Expected Result:** Error raised
- **Actual Result:** ValueError raised
- **Explanation:** Ensures system prevents invalid mission execution

---

Test Case 5: Successful Mission Execution

- **Modules Involved:** Mission, Crew

- **Scenario:** Mission assigned with all required roles present
- **Expected Result:** Mission successfully assigned
- **Actual Result:** Mission executed successfully
- **Explanation:** Confirms correct validation of crew roles

Test Case 6: Damaged Car Restriction

- **Modules Involved:** Inventory, Race
- **Scenario:** Attempt to use a damaged car in a race
- **Expected Result:** Error raised
- **Actual Result:** Error raised
- **Explanation:** Ensures damaged resources are not used

Test Case 7: Vehicle Repair and Reuse

- **Modules Involved:** Maintenance, Inventory
- **Scenario:** Damaged car is repaired and reused
- **Expected Result:** Car becomes usable
- **Actual Result:** Car successfully repaired

- **Explanation:** Validates maintenance module functionality

Errors and Fixes Identified

Error 1: Missing Data Validation in Race Module

- Issue: Race module did not validate driver and car data structures, which could lead to runtime errors
- Fix: Added validation to ensure required fields (role, name, damaged) are present

---

Error 2: Negative Inventory Balance

- Issue: Inventory allowed cash to become negative
- Fix: Added validation to prevent invalid balance updates

Error 3: Invalid Mission Input Handling

- Issue: Mission module allowed empty crew and invalid data structures
- Fix: Added validation for crew list, required roles, and member structure

---

Error 4: Invalid Registration Inputs

- Issue: Registration allowed empty names and undefined roles
- Fix: Added validation for name format and restricted roles to predefined values

---

Error 5: Invalid Role and Skill Assignment

- Issue: Crew module allowed invalid roles and unrealistic skill values
- Fix: Added validation for role correctness and skill range (0–100)